COMPUTER ARCHITECTURE & DIGITAL SYSTEM DESIGN

Project 02: Design and Verification of an 8-Bit 5-Stage Pipelined RISC CPU in Verilog

Comprehensive RTL Implementation, Hazard Resolution & Vivado Verification

Presenter: Truong Dong Hung
Roll Number: SE201121

Course: Computer Architecture & Digital System Design
Platform: Xilinx Vivado (XSim) | Target: Artix-7 FPGA (xc7a35t) | 100 MHz Clock

System-Level Architecture (System View)

High-Level CPU Overview & Testbench Interface

- **Core Topology:** Classic Harvard Architecture with separate 256x16-bit Program ROM (imem) and 256x8-bit Data RAM (dmem).
- **Primary I/O Ports:**
  - clk (100 MHz oscillator), rst (active-high synchronous reset).
  - Observability Outputs: halted (CPU finished), pc_out[7:0], flags_out[1:0] ({Z, C}), instr_done (valid retire pulse).
- **Standalone Execution:** Self-contained core; instruction memory preloaded via $readmemh("program.mem").
- **Closed-Loop Verification:** Top-level testbench monitors all internal pipeline registers, hazard events, and retired instructions cycle-by-cycle.

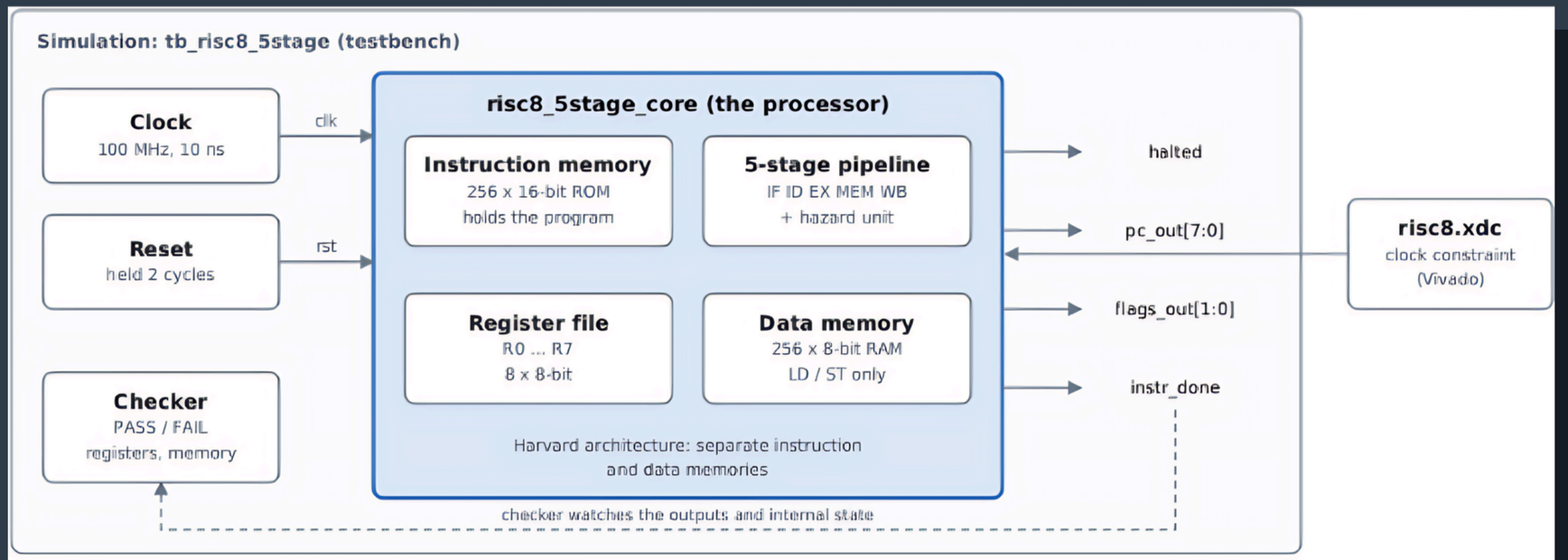


Figure 2. System view: the processor core inside its testbench, with the Vivado constraint file.

RTL HIERARCHY

RTL Modular Design & File Hierarchy

Structural Decomposition from Testbench to Leaf Cells

- **Top-Level Wrapper:** risc8_5stage_core.v manages the 4 pipeline registers and clock-synchronous controls.
- **Instantiated Submodules (6 Core Engines):**
  - imem.v (Stage 1 - IF): 256 x 16-bit ROM, asynchronous read.
  - control_unit.v (Stage 2 - ID): Purely combinational hardwired decoder & hazard helper signals.
  - regfile.v (Stage 2 - ID / Stage 5 - WB): 8 x 8-bit registers, 2 combinational read ports, 1 clocked write port, Write-Through Bypass.
  - hazard_unit.v (Stages 2/3 - ID/EX): Data forwarding engine & load-use stall detector.
  - alu.v (Stage 3 - EX): 8 arithmetic/logic operations, Zero & Carry flag generator.
  - dmem.v (Stage 4 - MEM): 256 x 8-bit RAM, combinational read, synchronous write.
- **Constraints & Environment:** risc8.xdc (100 MHz timing constraint) | Testbenches: tb_risc8_5stage.v, tb_dump.v.

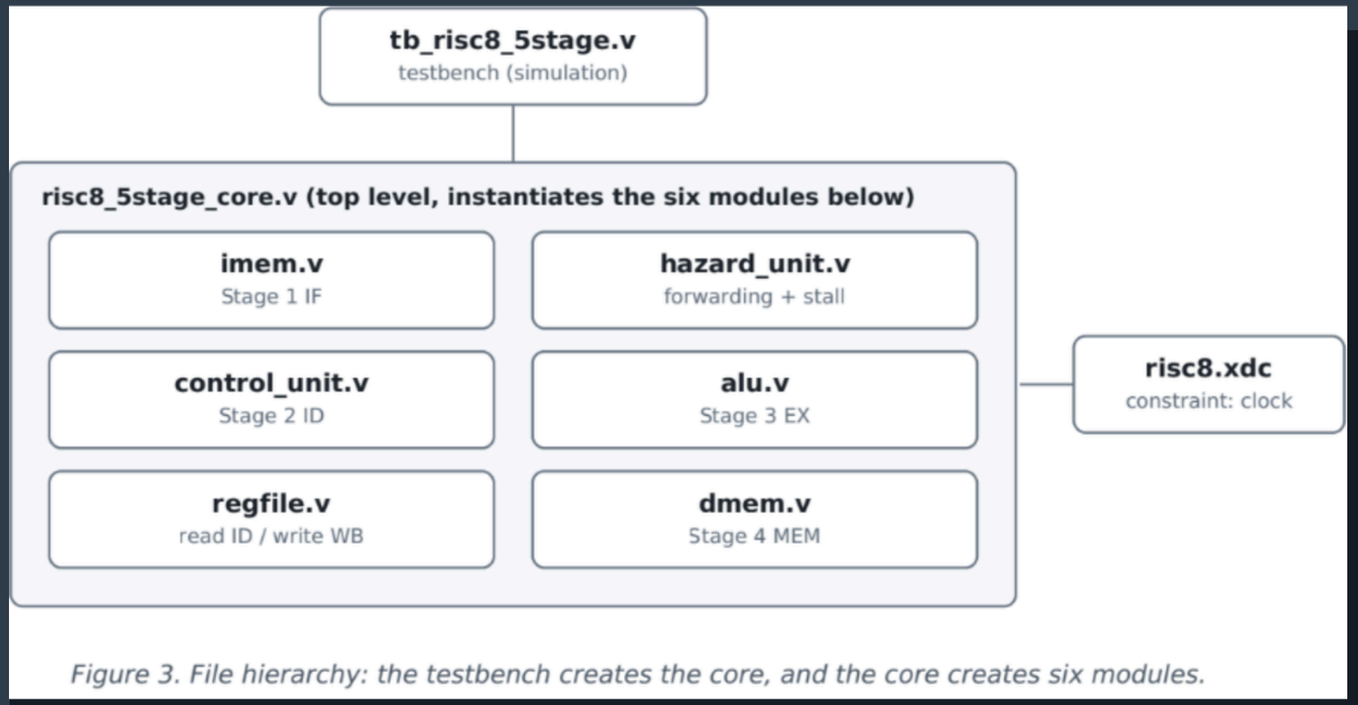


Figure 3. File hierarchy: the testbench creates the core, and the core creates six modules.

16-Bit Instruction Set Architecture

Uniform 16-bit Fixed-Length Instruction Encoding

Fixed-Position Decoding: The 4-bit Opcode [15:12] and destination register rd [11:9] are always in identical positions across formats, enabling instant hardware decoding before full instruction analysis.
Four Distinct Formats:

- **R-Type (3-Register Arithmetic & Logic):** [15:12] opcode | [11:9] rd | [8:6] rs1 | [5:3] rs2 | [2:0] 000 (ADD, SUB, AND, OR, XOR)
- **R1-Type (2-Register Shift & Memory):** [15:12] opcode | [11:9] rd | [8:6] rs1 | [5:0] 000000 (SHL, SHR, LD, ST)

- **I-Type (Immediate Operations):** [15:12] opcode | [11:9] rd | [8] 0 | [7:0] imm8 (LDI, ADDI)

- **J-Type (Control Flow & Branching):** [15:12] opcode | [11:8] 0000 | [7:0] addr8 (JMP, BZ, BNZ)

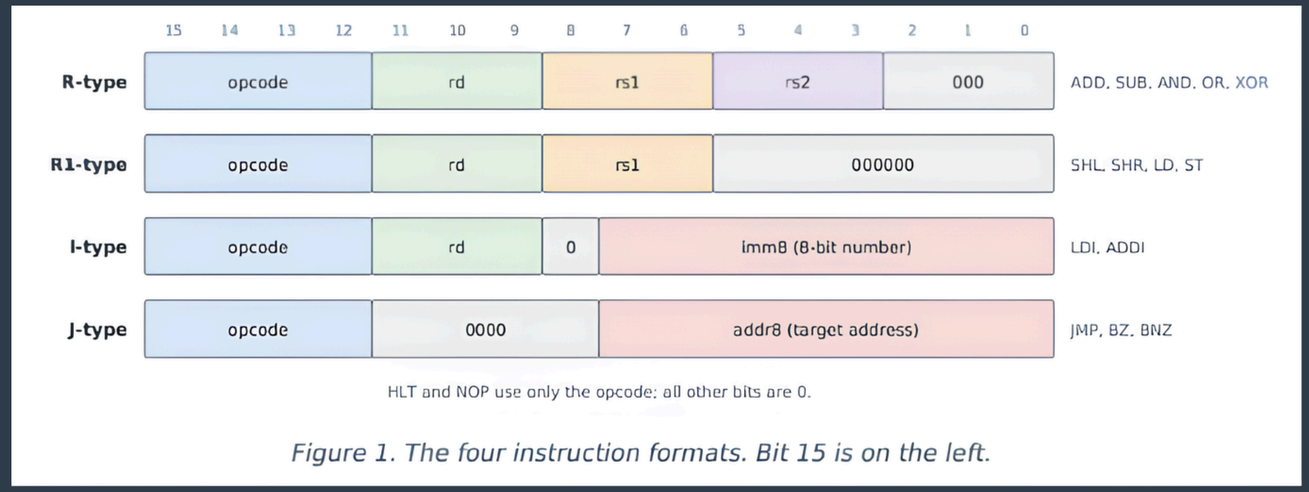


Figure 1. The four instruction formats. Bit 15 is on the left.

Special: NOP (0x0000) and HLT (0xF000) use only the opcode field.

Bit Field Breakdown & Semantic Definition

SPEC TABLE

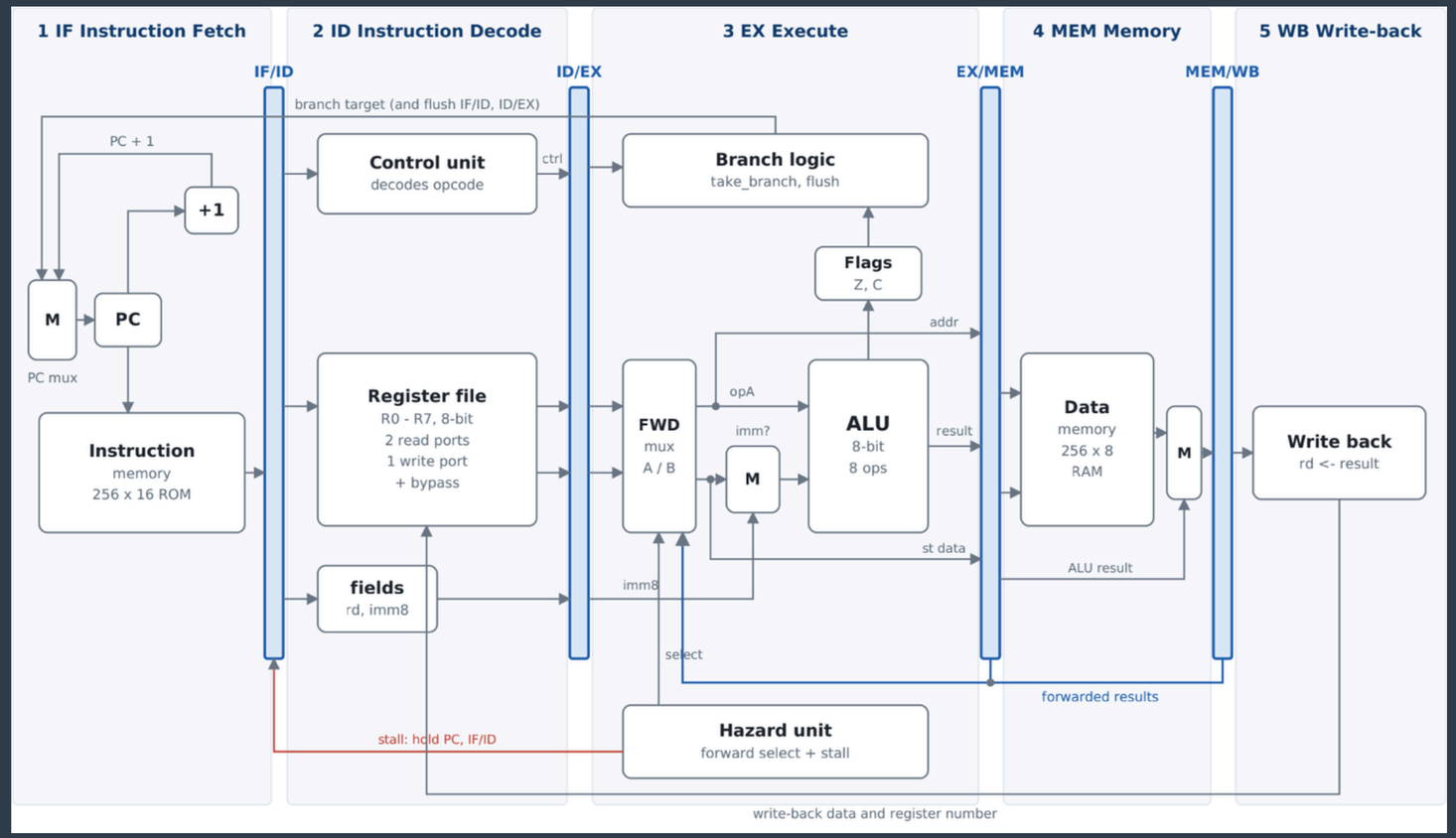
Bit Allocation Table & Hardware Usage

Field Name	Bit Range	Width	Hardware Function & Semantic Meaning
opcode	[15:12]	4 bits	Selects 1 of 16 instructions; drives the Control Unit lookup logic.
rd	[11:9]	3 bits	Destination register (R0-R7) for write-back; acts as store-data source register for ST.
rs1	[8:6]	3 bits	First source operand (ALU port A); holds base RAM pointer for LD and ST.
rs2	[5:3]	3 bits	Second source operand (ALU port B) for R-type instructions.
imm8 / addr8	[7:0]	8 bits	8-bit numerical constant for LDI/ADDI, or absolute target PC address for JMP/BZ/BNZ.

Complete 5-Stage Datapath Overview

Pipelined Execution Across Natural Hardware Boundaries

- **1. IF (Instruction Fetch):** PC drives imem address → Fetches 16-bit word → Computes PC + 1 → Latches into IF/ID.
- **2. ID (Instruction Decode):** Splits instruction fields → control_unit generates control signals → Dual-port regfile read → Latches into ID/EX.
- **3. EX (Execute):** Forwarding MUXes select newest operands → alu performs calculation → Evaluates Z and C flags → Branch logic resolves taken jumps → Latches into EX/MEM.
- **4. MEM (Memory Access):** Reads/writes dmem at address calculated in EX → Selects RAM data or ALU result → Latches into MEM/WB.
- **5. WB (Write-Back):** Writes final result into regfile at register rd → Completes instruction retirement.

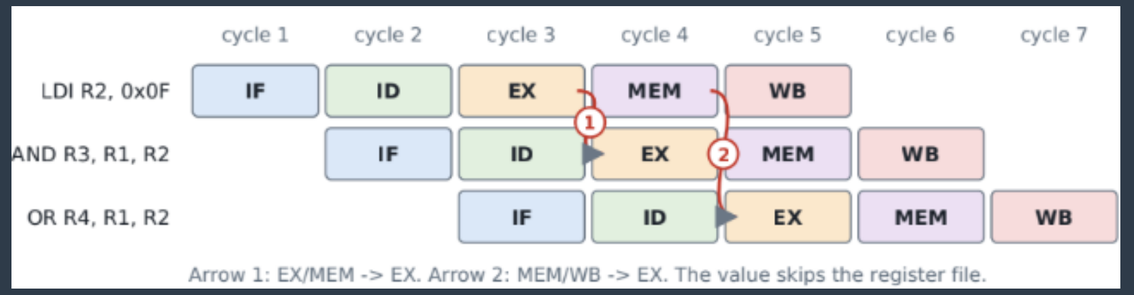


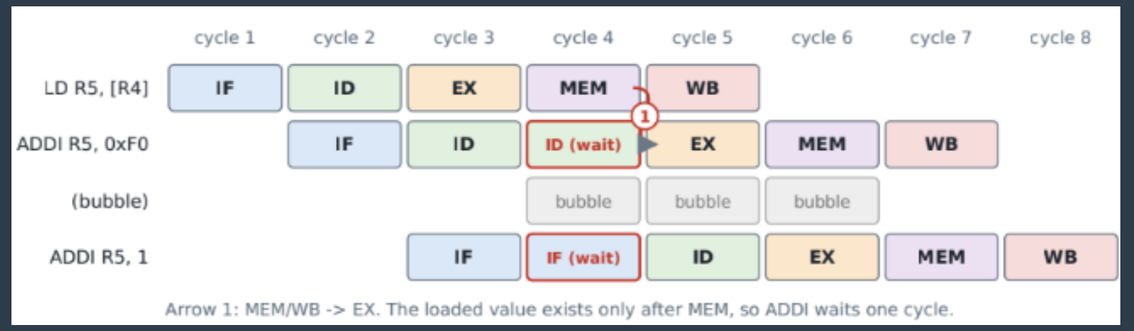
HAZARD RESOLUTION

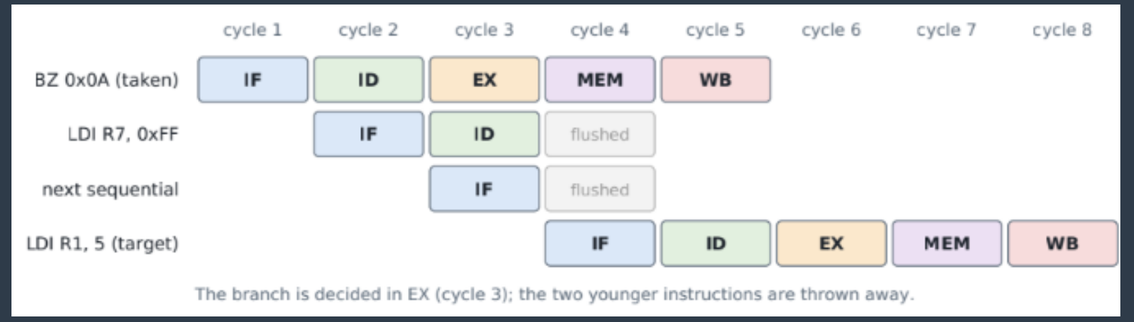
Comprehensive Hazard Handling

Hardware-Driven Conflict Resolution Strategies

- **Structural Hazards (Cost: 0 Cycles):**
  - Completely eliminated by the Harvard architecture (independent IMEM and DMEM) and a register file with separate read and write ports.
- **Data Hazards - RAW (Cost: 0 Cycles):**
  - **Distance 1:** Direct forwarding from EX/MEM to the ALU input in the EX stage.
  - **Distance 2:** Direct forwarding from MEM/WB to the ALU input in the EX stage.
  - **Distance 3:** Handled by the **Write-Through Bypass** circuit inside regfile.v when ID reads and WB writes in the same cycle.
- **Load-Use Data Hazards (Cost: 1 Cycle):**
  - Occurs when the instruction right after an LD needs its data. The hazard_unit automatically freezes PC and IF/ID, inserts 1 bubble into ID/EX, then forwards the result from MEM/WB.
- **Control Hazards (Cost: 2 Cycles):**
  - The branch decision is made in the EX stage (based on the updated Z flag). When a branch is taken (take_branch = 1), the hardware automatically **Flushes** the 2 wrongly fetched instructions in IF/ID and ID/EX.







CONTROL & INTERLOCK

Control Unit & Hazard Detection Unit Deep Dive

Intelligent Hardware Steering & Interlock Logic

Control Unit (control_unit.v)

- Pure hardwired combinational lookup table.
- Generates per-stage signals: `alu_op`, `alu_src_imm`, `reg_we`, `mem_we`, `flag_we`, `jump`, `br_z`, `br_nz`, `halt`.
- **Hazard Optimization:** Provides uses_a and uses_b signals so the Hazard Unit knows which port is actually read, completely eliminating false stalls and false forwarding.

Hazard Unit (hazard_unit.v)

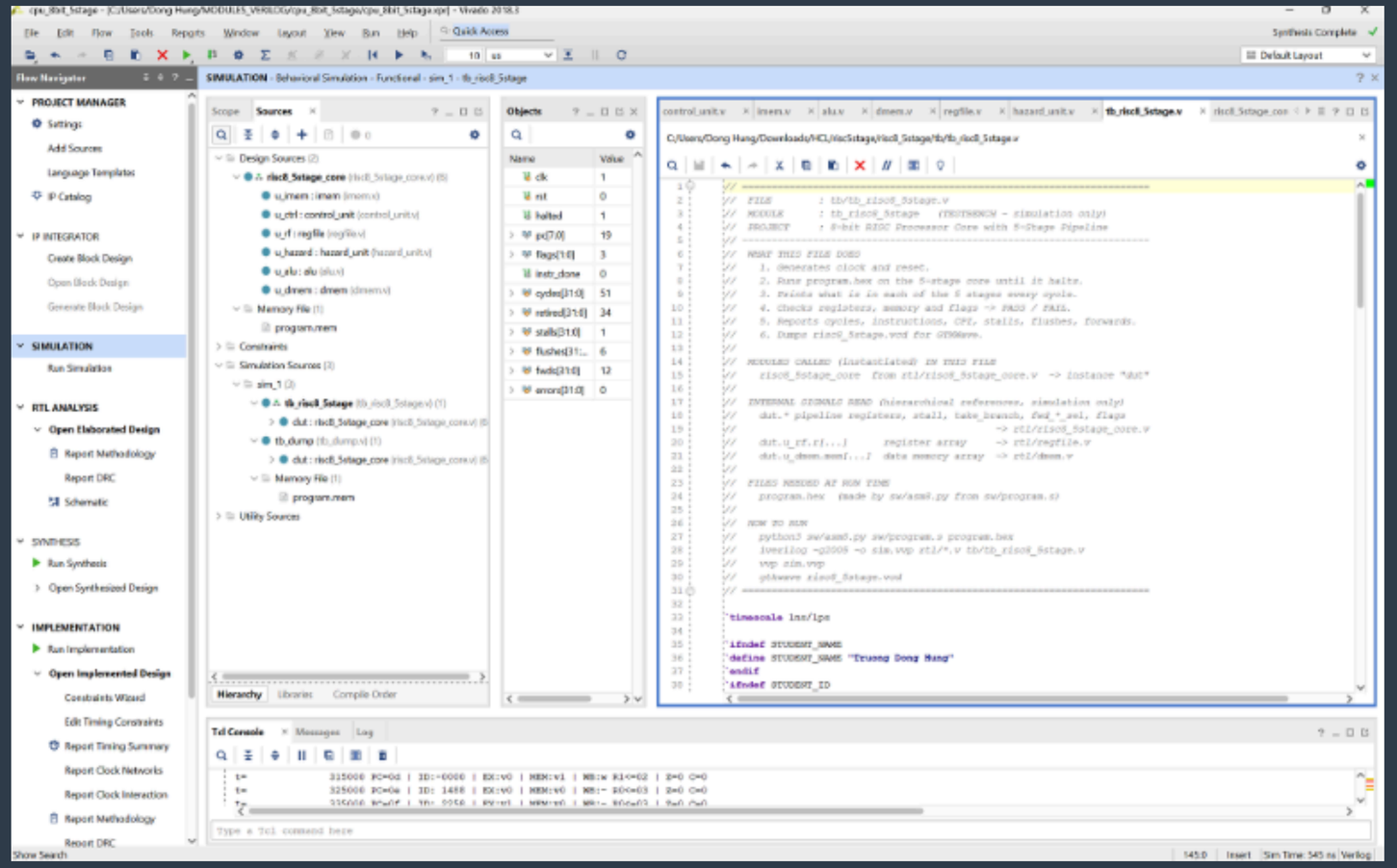
- Monitors register fields across all 4 stages (ID, EX, MEM, WB) simultaneously.
- Compares id_ra_addr/id_rb_addr with ex_rd when an LD is in EX → Triggers stall = 1.
- Prioritizes forwarding from the MEM stage over the WB stage to always receive the newest value.

Dual-Layer Verification Strategy

Structured Simulation & Golden Model Differential Testing

- **Layer 1: Directed Timing Testbench (tb_risc8_5stage.v):**
  - Cycle-by-cycle execution trace logging through the pipeline stages.
  - Automatic statistics: Cycles, Instructions retired, CPI, Stalls, Flushes, Forwards.
  - Self-checking PASS/FAIL on final state for the 8 registers, RAM cells and the status flag pair.

- **Layer 2: Golden Model Regression (tb_dump.v + random_test.py):**
  - Automatically runs 500 randomly generated programs.
  - Dumps the full architectural state (8 registers, 256 bytes of memory) to dump.txt after HLT.
  - Bit-exact match against the reference software simulator (Python ISS).

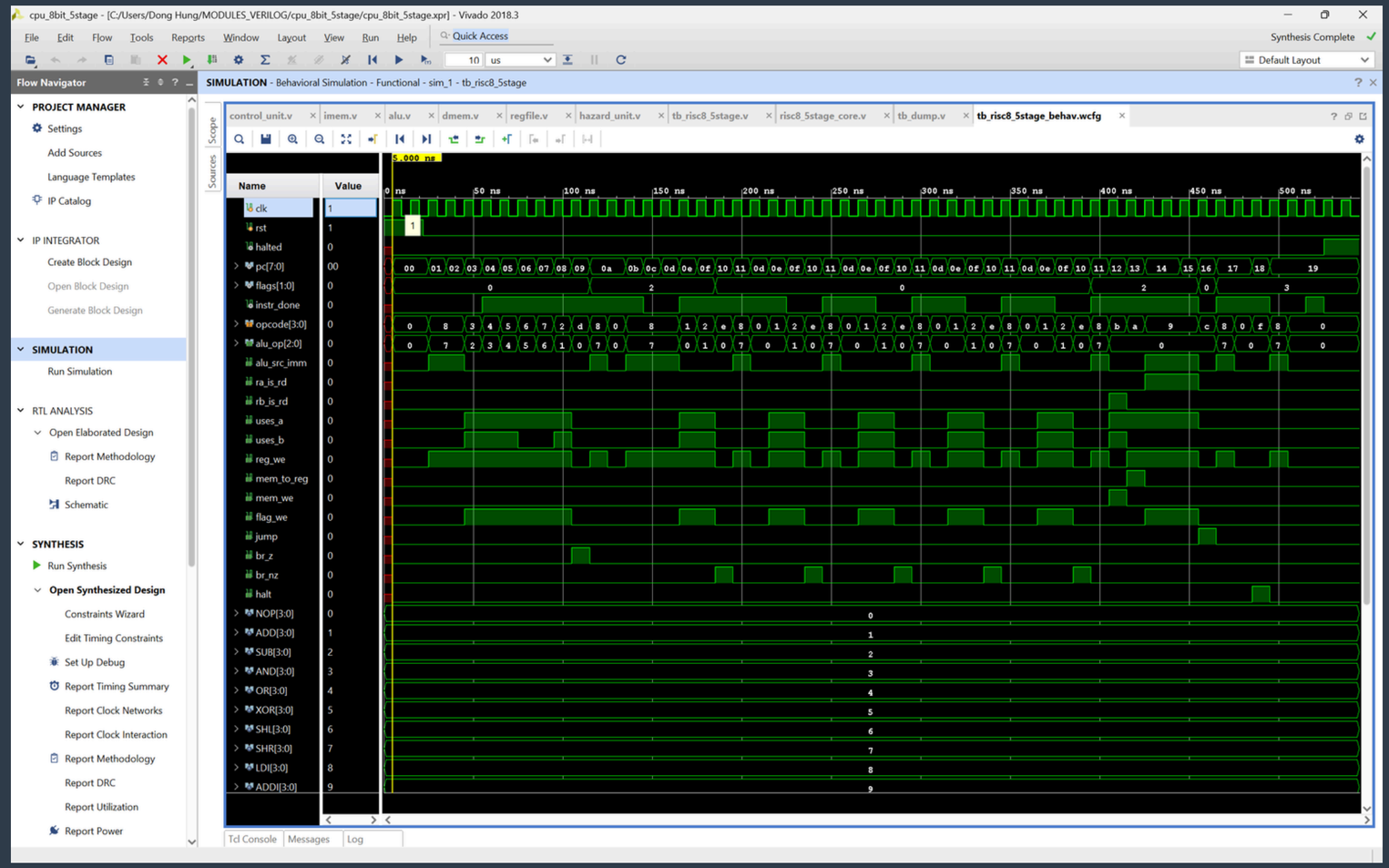


Experimental Results & Vivado Waveform

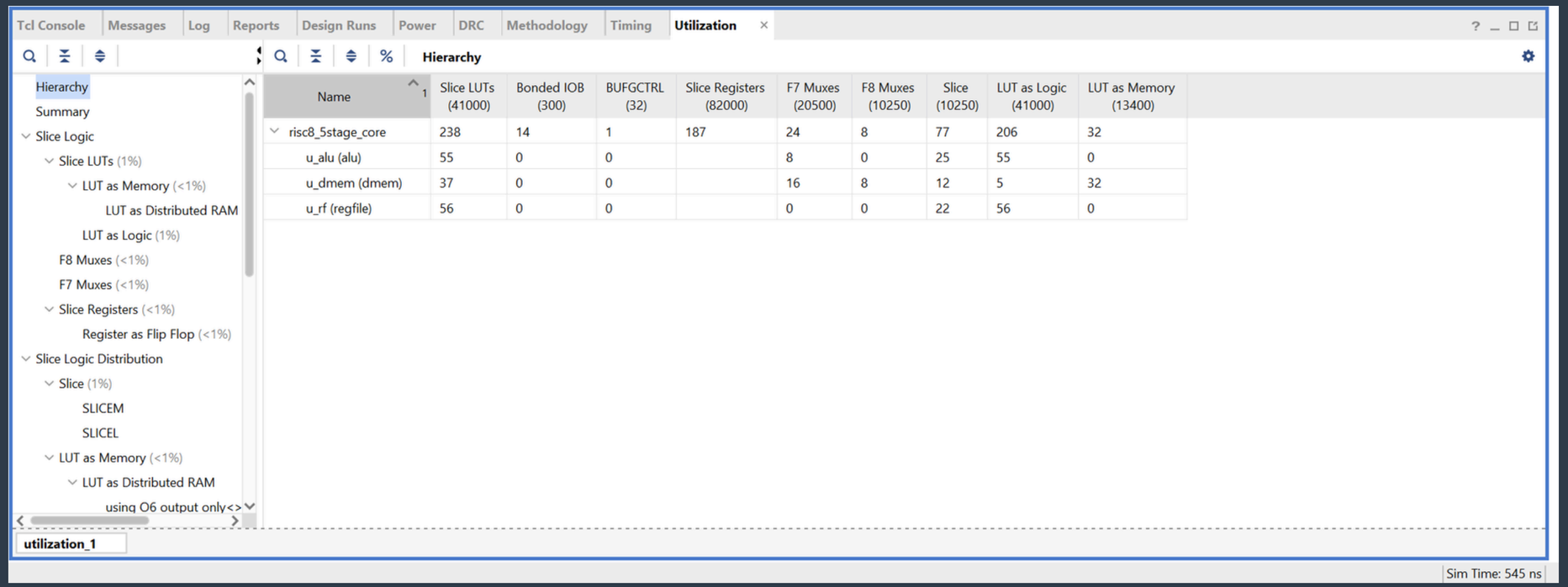
Simulation Log & Architectural Verification Metrics

Simulation Trace Highlights:

- t = 125ns: BZ skip1 branch taken → triggers BRANCH->flush 2, discarding the wrong instruction LDI R7, 0xFF.
- t = 205ns - 355ns: Sum loop Σ(i=1..5) i = 15 (0x0F); BNZ branches correctly 4 times, the 5th falls through.
- t = 435ns: LD followed by ADDI triggers exactly **1 STALL cycle**, then forwards data successfully.
- t = 465ns - 475ns: 0xFF + 1 = 0x00 overflows, correctly updating flags Z=1, C=1.
- t = 545ns: CPU halts safely at HLT after draining the pipeline (Pipeline Drain).
- **`Final Checks:`** `R1=00, R2=0F, R3=01, R4=20, R5=00, R6=66, R7=00, MEM[20]=0F, Flags Z=1 C=1 →` **`100% PASS.`**



FPGA Resource Utilization



Name	Slice LUTs (41000)	Bonded IOB (300)	BUFGCTRL (32)	Slice Registers (82000)	F7 Muxes (20500)	F8 Muxes (10250)	Slice (10250)	LUT as Logic (41000)	LUT as Memory (13400)
risc8_5stage_core	238	14	1	187	24	8	77	206	32
u_alu (alu)	55	0	0		8	0	25	55	0
u_dmem (dmem)	37	0	0		16	8	12	5	32
u_rf (regfile)	56	0	0		0	0	22	56	0

- 238 Slice LUTs (~0.58%)
- 187 Slice Registers
- 32 Memory LUTs